

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
DEPARTAMENTO DE INFORMÁTICA E MATEMÁTICA APLICADA

XANIM'S VAULT

Sistema automatizado de alimentação para felinos

Relatório final apresentado à disciplina de
DIM0616 - Sistemas Embarcados,
Docente Mônica Magalhães Pereira.

Andriel Vinicius de Medeiros Fernandes
Lucas Vinicius Dantas de Medeiros
María Paz Marcato

Natal/RN 2026

Sumário

1	Introdução	3
2	Descrição do sistema	3
2.1	Objetivo	3
2.2	Usuário	3
2.3	Visão geral da arquitetura	3
2.4	Entradas, processamento e saídas	4
2.5	Cálculo do plano alimentar	4
2.6	Escopo do protótipo	5
2.7	Fora do escopo	5
3	Estado da arte	5
4	Requisitos funcionais e não funcionais	6
4.1	Requisitos funcionais	6
4.2	Requisitos não funcionais	7
4.3	Regras de sistema	8
5	Modelagem	8
5.1	Diagrama de componentes	8
5.2	Diagrama de blocos do hardware	9
5.3	Diagrama de fiação	10
5.4	Máquina de estados	10
5.4.1	Parte 1: Cadastro e identificação inicial	11
5.4.2	Parte 2: Detecção de NFC e tratamento de intrusos	12
5.4.3	Parte 3: Liberação e monitoramento de consumo	13
5.4.4	Parte 4: Atualização de configurações	13
5.5	Modelo de dados	14
6	Implementação	15
6.1	Tecnologias utilizadas	15
6.2	Dificuldades encontradas	15
7	Conclusão	15
8	Referências	16

1 Introdução

A alimentação adequada dos felinos é fundamental para o seu desenvolvimento saudável. Em residências com mais de um gato em condições distintas como por exemplo, animais castrados e não castrados, filhotes ou gatos em diferentes fases etárias, torna necessário oferecer porções e tipos de ração específicos para cada animal, uma vez que as necessidades nutricionais variam consideravelmente entre eles. Gatos castrados, por exemplo, tendem a ter metabolismo mais lento e exigem dietas com menor teor calórico em comparação aos não castrados.

Entretanto, o comportamento natural dos felinos dificulta o controle alimentar individualizado. Por serem animais independentes e territorialistas, é comum que um gato consuma a ração destinada a outro, especialmente quando este demora mais para se alimentar ou se ausenta momentaneamente do pote. Esse consumo indevido de alimento gera desequilíbrios nutricionais que podem comprometer a saúde dos animais ao longo do tempo.

Além disso, gatos devem se alimentar em múltiplos horários ao longo do dia, de modo a evitar tanto o acúmulo de calorias desnecessário quanto a ansiedade associada à concentração da alimentação em apenas um ou dois momentos do dia. Soluções existentes no mercado, como comedouros automáticos com horário programado, apresentam limitações relevantes: a liberação de ração em horários fixos, sem qualquer mecanismo de controle de acesso, não impede que outro animal da casa consuma a porção alheia, e o esquecimento humano de repor a ração manualmente pode resultar em períodos de privação alimentar.

Diante desse cenário, o presente projeto propõe o desenvolvimento do Xanim's Vault, um dispenser inteligente e automático de ração, capaz de identificar o animal por meio de tecnologia NFC, calcular e liberar porções individualizadas ao longo do dia e impedir o acesso de outros felinos à ração de um gato específico.

2 Descrição do sistema

2.1 Objetivo

O sistema tem como objetivo gerenciar a dieta de cada felino cadastrado de forma automatizada e inteligente, atuando no cálculo das porções diárias, na liberação da ração nos horários programados e no mecanismo de defesa contra o consumo por animais não autorizados.

2.2 Usuário

O principal usuário do sistema é o pai/mãe de pet, preocupado com a saúde alimentar do seu gato e com a possibilidade de outros animais da casa interferirem em sua dieta.

2.3 Visão geral da arquitetura

O Xanim's Vault é composto por quatro grandes blocos, que se comunicam conforme ilustrado no diagrama de componentes (ver Seção 5.1):

- **Aplicativo (UI)**: aplicação para cadastro do animal e visualização de estatísticas de consumo, desenvolvida em MIT App Inventor;
- **Backend / CMS (Strapi)**: responsável por armazenar os dados de gatos, dietas, horários e registros, além de calcular o plano alimentar;
- **Banco de dados (SQLite)**: banco de dados leve utilizado pelo Strapi para persistência das informações;
- **Broker MQTT (Mosquitto)**: intermediário de comunicação publish/subscribe entre o backend e o sistema embarcado;
- **Sistema embarcado (ESP32)**: responsável pela leitura de NFC, pesagem da ração, liberação das porções, bloqueio do pote e envio de eventos.

A comunicação entre os módulos ocorre da seguinte forma:

- **Aplicativo ↔ Backend**: protocolo HTTP/REST;
- **Backend ↔ Sistema embarcado**: protocolo MQTT, via broker Mosquitto.

2.4 Entradas, processamento e saídas

Entradas do sistema:

1. Cadastro do animal (nome, peso, data de nascimento, status de castração e identificador NFC);
2. Leitura do tempo corrente, via módulo RTC;
3. Detecção da tag NFC e leitura da célula de carga (balança).

Processamento:

1. Criação do plano alimentar pelo backend (CMS), a partir dos dados cadastrais do gato;
2. Processos do sistema embarcado: alerta sonoro (buzzer), liberação da ração e monitoramento de consumo.

Saídas do sistema:

1. Abertura/fechamento do pote de ração;
2. Envio de estatísticas e eventos ao servidor.

2.5 Cálculo do plano alimentar

O cálculo da quantidade diária de ração segue as diretrizes nutricionais da AAHA (2021 AAHA Nutrition and Weight Management Guidelines for Dogs and Cats), utilizando as fórmulas de necessidade energética em repouso (RER) e necessidade energética de manutenção (MER):

$$\text{RER} = \text{peso}_{\text{kg}}^{0.75} \times 70$$

$$\text{MER} = \text{RER} \times \text{fator do estágio de vida}$$

O fator do estágio de vida varia conforme a condição do animal (por exemplo, entre 1,2 e 1,4 para adultos castrados, e entre 1,4 e 1,6 para adultos intactos). A partir do valor de MER (em kcal/dia), o sistema converte a energia necessária em gramas de ração, considerando a concentração calórica da ração de referência (Quatree Gourmet):

- Filhote: 3820 kcal/kg (3,82 kcal/g);
- Adulto: 3800 kcal/kg (3,80 kcal/g);
- Adulto castrado: 3705 kcal/kg (3,70 kcal/g).

A quantidade diária calculada é então dividida entre os horários de alimentação configurados (3, 4 ou 6 horários, sendo 4 o padrão). Durante a liberação, a ração é dispensada aos poucos, permitindo que o ESP32 monitore o peso na balança em tempo real e interrompa a liberação assim que a porção programada for atingida, com margem de erro de ± 5 g.

2.6 Escopo do protótipo

Para a entrega ao final do semestre, foi definido que a simulação seria feita de forma física, mas não necessariamente com todos os componentes em condição de produção:

- O servidor web (CMS) roda localmente, utilizando Strapi;
- A “coleira” do gato é representada apenas pelo chip NFC;
- Elementos de papelão são utilizados para representar potes e travas.

2.7 Fora do escopo

Os seguintes itens foram explicitamente definidos como fora do escopo do projeto:

- Reconhecimento facial de animais e uso de câmeras;
- Integração com serviços externos;
- Controle simultâneo de múltiplos gatos ou múltiplos potes;
- Uso de inteligência artificial;
- Controle de estoque de ração;
- Aplicativo comercial / sistema em produção;
- Atualização de firmware remota.

3 Estado da arte

Durante a etapa de pesquisa, foram identificados diversos trabalhos e produtos relacionados a alimentadores automáticos de animais, que serviram de referência para a definição do escopo e das funcionalidades do Xanim's Vault:

Trabalho / Produto	Descrição
Comedouro automático para cães em situação de rua	Projeto voltado ao apoio de ONGs no cuidado de cães resgatados, com liberação automatizada de ração.
Alimentador automático para nutrição de gatos com ESP32 (UFRGS)	Trabalho de conclusão que utiliza ESP32 para controle da alimentação de felinos.
ComunaPet - Solução inteligente para alimentação de pets via IoT	Bebedouro automatizado e comedouro manual para ani-

	mais de rua, com monitoramento via aplicativo PetFriendly.
Dispenser de ração automático com Arduino	Projeto de eletrônica com liberação automatizada de ração utilizando Arduino.
Alimentador com ESP32, MQTT e dashboards (UFERSA)	Sistema embarcado que utiliza MQTT para comunicação e dashboards para visualização dos dados de alimentação.
Coleira com tecnologia NFC	Trabalho relacionado ao uso de tags NFC para identificação individual de animais.

Além desses trabalhos acadêmicos, foram analisados produtos comerciais, como comedouros automáticos com conectividade Wi-Fi e controle via aplicativo (ex.: linha Petlibro) e comedouros/bebedouros duplos de baixo custo. Em geral, esses produtos comerciais permitem programação de horários e porções, mas não oferecem identificação individual do animal nem mecanismo de bloqueio de acesso para outros pets da casa, diferencial central proposto pelo Xanim's Vault.

4 Requisitos funcionais e não funcionais

Nesta seção são apresentados os requisitos finais do sistema, já refinados a partir da proposta inicial.

4.1 Requisitos funcionais

RF01 Cadastro do animal. O aplicativo deve permitir o cadastro de um gato contendo nome, peso, idade, status de castração e identificador NFC.

RF02 Geração do plano alimentar. O CMS deve calcular automaticamente a quantidade diária de ração com base nas características cadastradas do gato, utilizando por padrão 4 horários.

RF03 Configuração de horários. O aplicativo deve permitir configurar 3, 4 ou 6 horários de alimentação diária.

RF04 Distribuição das porções. O CMS deve dividir automaticamente a quantidade diária de ração entre os horários configurados.

RF05 Sincronização com o sistema embarcado. O CMS deve enviar ao sistema embarcado, via MQTT, os horários de alimentação, a quantidade de ração por porção e o identificador NFC autorizado.

RF06 Persistência local. O sistema embarcado deve armazenar em memória flash o identificador NFC, os horários de alimentação e a quantidade por porção.

RF07 Atualização de dados. O sistema embarcado deve atualizar os dados armazenados localmente sempre que novas configurações forem recebidas do CMS.

RF08 Liberação automática da ração. O sistema embarcado deve liberar automaticamente a porção programada nos horários definidos, junto de um alerta sonoro via buzzer.

RF09 Controle da quantidade liberada. O sistema embarcado deve utilizar uma balança para medir a quantidade de ração liberada, interrompendo a liberação quando a quantidade programada for atingida.

RF10 Identificação NFC. O sistema embarcado deve detectar tags NFC posicionadas a até 4 cm do leitor.

RF11 Validação de acesso. O sistema embarcado deve comparar a tag NFC detectada com a tag cadastrada. Caso válida, o acesso ao pote deve ser permitido; caso contrário, o pote deve ser bloqueado e um alerta enviado ao CMS.

RF12 Monitoramento do consumo. O sistema embarcado deve monitorar o peso restante da ração liberada utilizando a célula de carga, registrando o peso da porção logo após a liberação e realizando leituras periódicas e estabilizadas do peso restante. O consumo é classificado como:

- Consumo total: peso restante inferior a 10% da porção liberada;
- Consumo parcial: peso restante entre 11% e 90% da porção liberada;
- Consumo não realizado: peso restante superior a 90% da porção liberada.

RF13 Registro de eventos. O sistema embarcado deve enviar ao CMS o horário da alimentação, a quantidade liberada, o status do consumo, a identificação NFC detectada e eventuais alertas de acesso inválido.

RF14 Operação offline. O sistema embarcado deve continuar operando com base nos dados salvos na memória flash quando estiver sem conexão com a internet ou com o broker MQTT.

RF15 Visualização de estatísticas. O aplicativo deve permitir visualizar os horários em que o gato se alimentou, o consumo parcial ou completo, as tentativas de acesso por outros gatos e os alertas gerados pelo sistema.

4.2 Requisitos não funcionais

RNF01 Precisão da liberação. O sistema embarcado deve possuir margem de erro máxima de ± 5 g na liberação da ração.

RNF02 Tempo de resposta do bloqueio. O mecanismo de fechamento do pote deve responder em até 1 segundo após a identificação de uma tag NFC inválida.

RNF03 Persistência de dados. As informações armazenadas em memória flash devem permanecer disponíveis mesmo após reinicialização inesperada ou queda de energia do sistema embarcado.

RNF04 Tempo de detecção NFC. A tag NFC deve ser detectada em no máximo 1 segundo após a aproximação ao leitor.

RNF05 Comunicação. A comunicação entre o servidor (CMS) e o sistema embarcado deve utilizar o protocolo MQTT, via broker Mosquitto, enquanto a comunicação entre o aplicativo e o CMS deve utilizar o protocolo HTTP.

RNF06 Funcionamento offline. O sistema embarcado deve manter o funcionamento básico mesmo sem conexão com a internet.

RNF07 Sincronização de configurações. Novas configurações recebidas via MQTT devem sobrescrever os dados armazenados anteriormente na memória flash.

4.3 Regras de sistema

- **RN01:** A soma das porções distribuídas ao longo do dia deve ser igual à quantidade diária calculada pelo CMS.
- **RN02:** Somente a tag NFC cadastrada para o gato pode acessar o seu pote de ração.
- **RN03:** A liberação da ração deve ocorrer apenas nos horários programados.
- **RN04:** Cada evento de alimentação deve gerar um registro armazenado no CMS.

5 Modelagem

Esta seção apresenta os diagramas elaborados durante a fase de projeto, com a respectiva explicação de cada um.

5.1 Diagrama de componentes

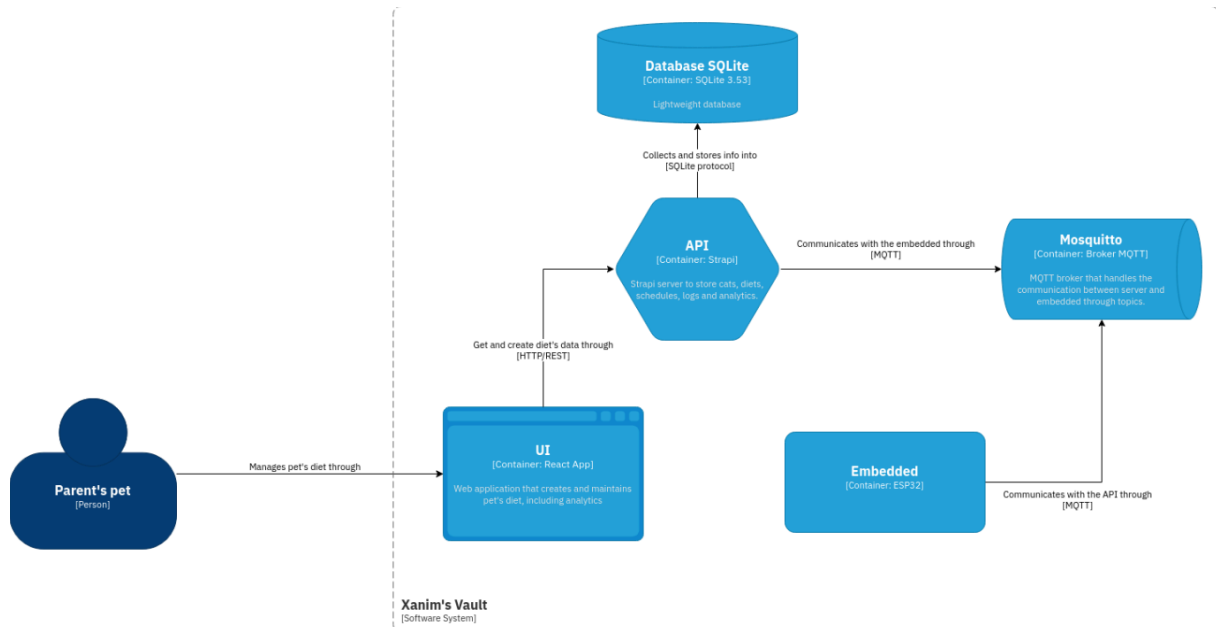


Figura 1: Diagrama de componentes do Xanim's Vault, no estilo C4, mostrando a interação entre o usuário (pai/mãe do pet), a aplicação (UI), o CMS (Strapi), o banco de dados (SQLite), o broker MQTT (Mosquitto) e o sistema embarcado (ESP32).

O diagrama de componentes descreve a arquitetura de alto nível do sistema. O usuário interage com a UI, que se comunica com o backend CMS via HTTP/REST para recuperar e atualizar os dados da dieta do animal. O CMS armazena as informações no banco SQLite e publica os dados de configuração para o sistema embarcado através do broker MQTT (Mosquitto). O sistema embarcado, por sua vez, envia ao broker os eventos de alimentação, consumo e alertas de intrusão, que são consumidos pelo CMS.

5.2 Diagrama de blocos do hardware

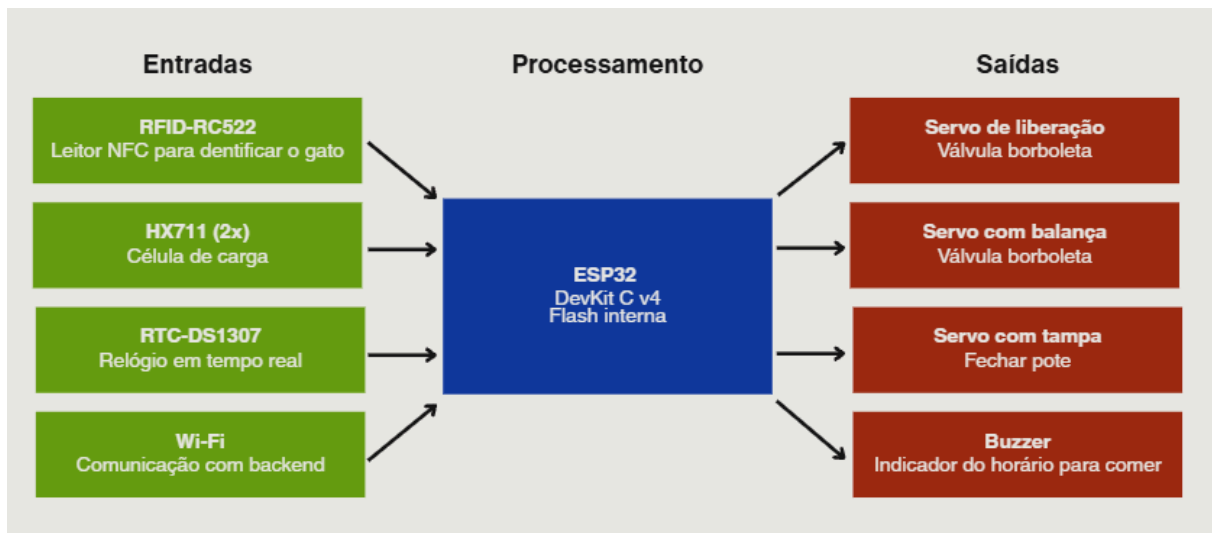


Figura 2: Diagrama de blocos do hardware, dividido em entradas, processamento e saídas.

O diagrama de blocos organiza o hardware em três grupos:

- **Entradas:** leitor NFC (RFID-RC522), duas células de carga (HX711), módulo RTC (DS1307/DS3231) e módulo Wi-Fi;
- **Processamento:** microcontrolador ESP32 DevKit C v4, com memória flash interna;
- **Saídas:** servo de liberação (válvula borboleta), servo com balança (válvula borboleta), servo com tampa (fechamento do pote) e buzzer (indicador sonoro do horário de alimentação).

5.3 Diagrama de fiação

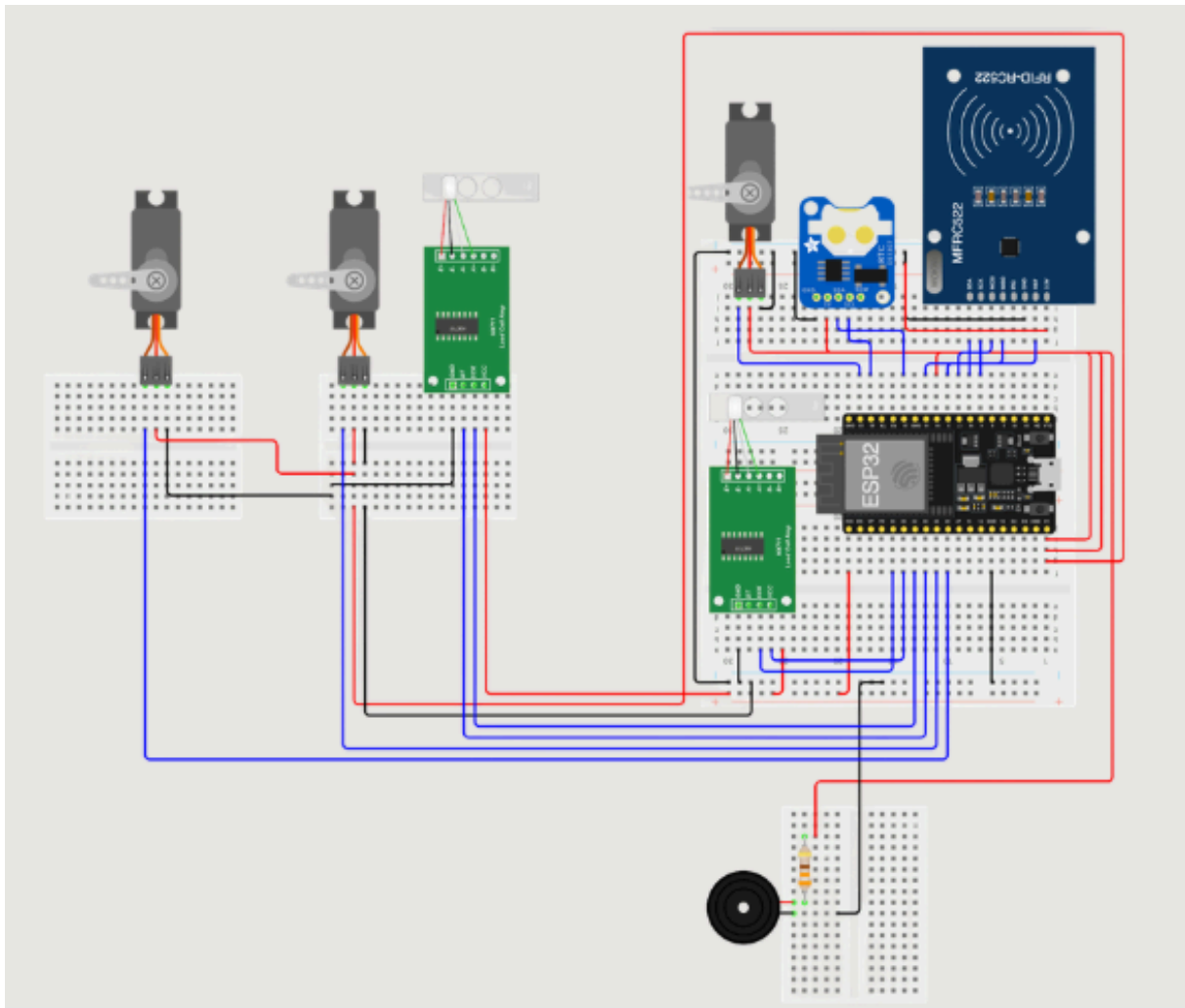


Figura 3: Diagrama de fiação da montagem em protoboard, mostrando a conexão do ESP32 aos servomotores, ao módulo RFID-RC522, às células de carga (via módulos HX711) e ao buzzer.

Este diagrama detalha as conexões físicas entre o ESP32 e os demais componentes: os servomotores responsáveis pela liberação da ração e pelo fechamento do pote, o módulo RFID-RC522 para leitura das tags NFC, os módulos HX711 conectados às células de carga, e o buzzer utilizado como indicador sonoro.

5.4 Máquina de estados

O comportamento do sistema embarcado foi modelado como uma máquina de estados dividida em quatro partes.

5.4.1 Parte 1: Cadastro e identificação inicial

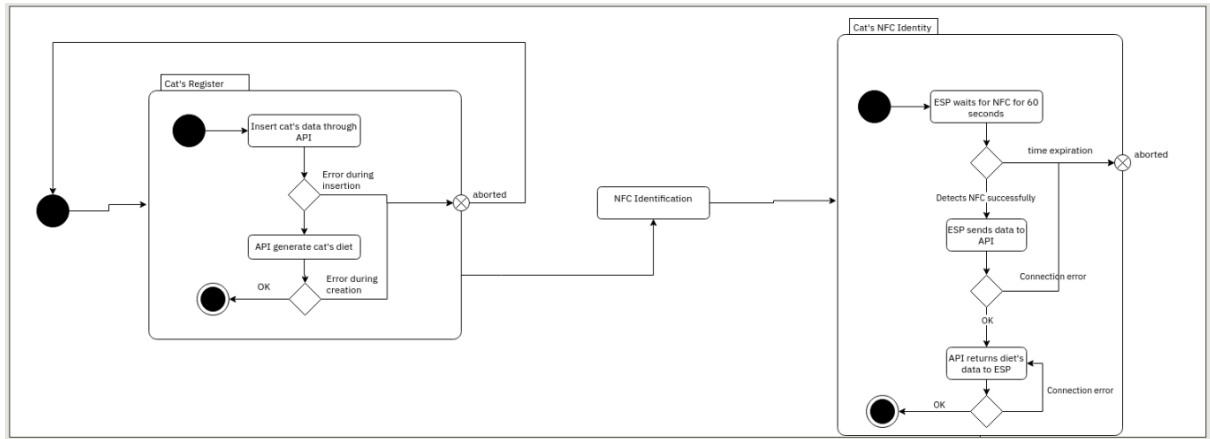


Figura 4: Etapa inicial: registro do gato, leitura de NFC e envio de dados ao embarcado.

Nesta primeira etapa, os dados do gato são inseridos via API. Caso não haja erro na inserção, o CMS gera automaticamente o plano alimentar. Em seguida, o ESP32 aguarda a detecção de um NFC por até 60 segundos; ao detectar, envia os dados à API, que retorna as informações da dieta para o embarcado.

5.4.2 Parte 2: Detecção de NFC e tratamento de intrusos

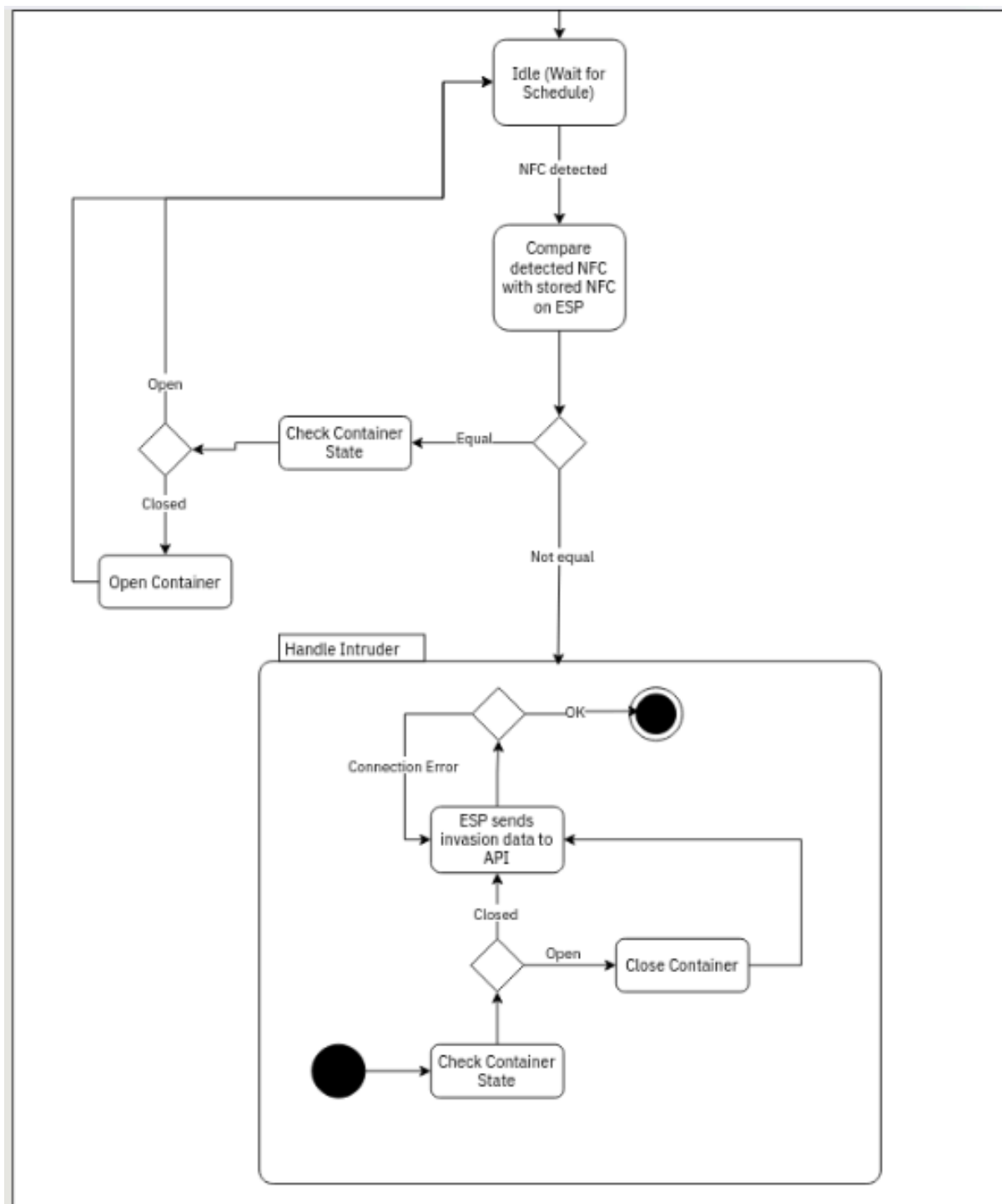


Figura 5: Etapa de detecção de NFC: verificação da tag detectada e envio de alerta de intruso, caso necessário.

Ao detectar uma tag NFC, o sistema compara-a com a tag armazenada. Se a tag corresponder, o estado do pote (aberto/fechado) é verificado e, se necessário, o pote é aberto. Caso a tag não corresponda, o sistema trata o evento como uma intrusão: verifica o estado do pote, fecha-o caso esteja aberto e envia os dados da invasão à API.

5.4.3 Parte 3: Liberação e monitoramento de consumo

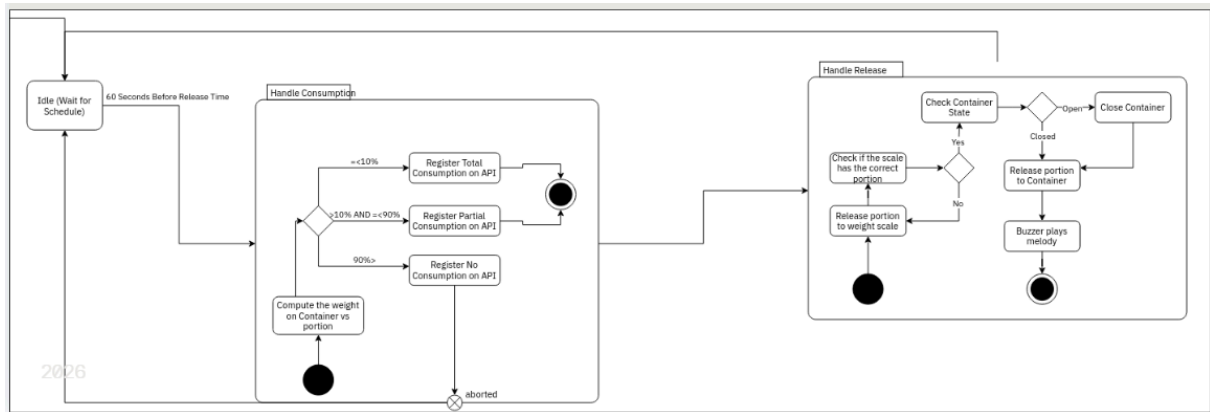


Figura 6: Etapa de liberação de ração: registro do consumo e liberação de nova porção, se necessário.

Sessenta segundos antes do próximo horário de liberação, o sistema calcula o peso do recipiente em relação à porção liberada anteriormente e classifica o consumo (total, parcial ou não realizado), registrando o resultado na API. Em seguida, é verificado se a balança contém a porção correta; caso não, uma nova porção é liberada até o recipiente atingir a quantidade programada. Por fim, o pote é fechado e o buzzer soa uma melodia indicando a conclusão do ciclo.

5.4.4 Parte 4: Atualização de configurações

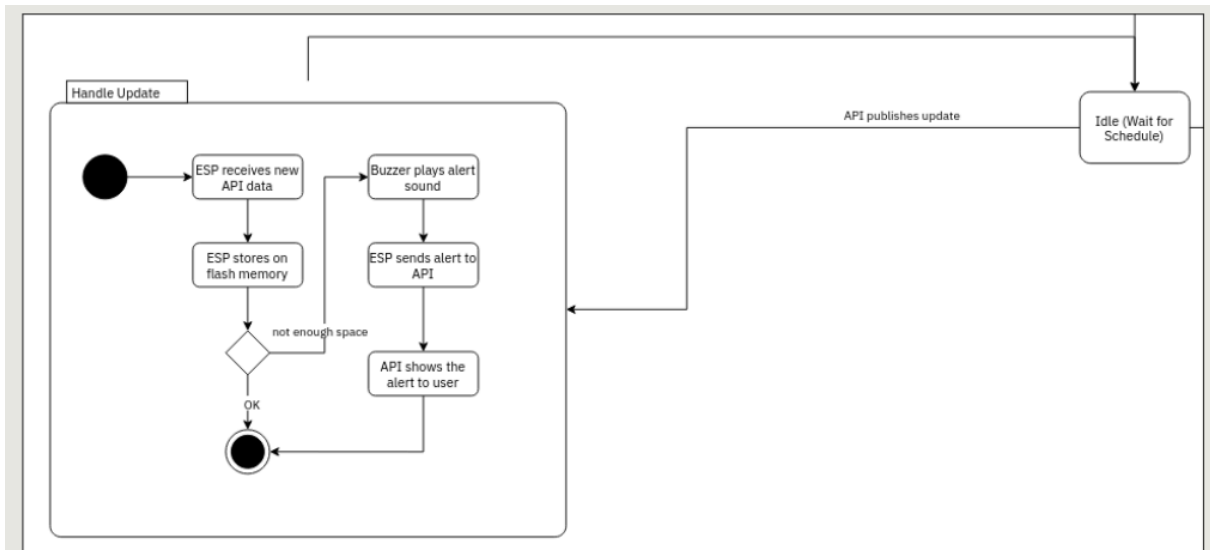


Figura 7: Etapa de atualização do embarcado: recepção de novos dados e persistência na flash, com envio de alertas em caso de falha.

Sempre que o CMS publica uma atualização, o ESP32 recebe os novos dados e os armazena em memória flash. Caso não haja espaço suficiente, o buzzer emite um alerta sonoro e o ESP32 envia um alerta à API, que o exibe ao usuário; caso a gravação seja bem-sucedida, o sistema retorna ao estado de espera (idle).

5.5 Modelo de dados



Figura 8: Modelo físico das entidades do banco de dados (SQLite).

O modelo de dados é composto pelas seguintes entidades principais:

- **User**: usuário responsável pelo cadastro (nome, login, senha);
- **Cat**: dados do gato (nome, data de nascimento, peso, status de castração, identificador NFC), relacionado ao usuário e ao fator de estágio de vida;
- **LifeStageFactor**: fator multiplicador utilizado no cálculo do MER, de acordo com a condição do animal;
- **Ration**: dados da ração cadastrada (descrição, marca, kcal por grama, categoria);
- **Diet**: dieta de um gato, associando o animal a uma ração e à porção calculada;
- **DietSchedule**: horários de alimentação associados a uma dieta;
- **Consumption**: registros de consumo do animal, associados ao tipo de consumo (total, parcial, não realizado) e ao horário de alimentação;
- **ConsumptionTypeId**: tipos de consumo possíveis;
- **IntrusionAlert**: registros de tentativas de acesso por NFC não autorizado.

6 Implementação

Esta seção deve ser finalizada Esta seção deve deixar claro: ● quais requisitos funcionais foram implementados; ● quais requisitos funcionais não foram implementados; ● quais tecnologias de hardware e software foram utilizadas; ● quais dificuldades foram encontradas durante o desenvolvimento.

6.1 Tecnologias utilizadas

Hardware:

- Microcontrolador ESP32 (DevKit C v4);
- Módulo NFC PN532 V3 / leitor RFID-RC522;
- Células de carga de 50 kg (2x ou 4x) com módulo HX711;
- Módulo RTC DS1307 / DS3231 (Real Time Clock, I²C);
- Micro servo motor Tower Pro MG90S (180°) - liberação da ração;
- Micro servo motores SG90 (9g) - válvula da balança e tampa do pote;
- Buzzer ativo 5V;
- Elementos de papelão para representação física do pote e das travas.

Software:

- Linguagem C/C++ para programação do ESP32 (firmware embarcado, via Arduino/ESP-IDF);
- Protocolo MQTT, com broker Mosquitto, para comunicação entre backend e embarcado;
- Protocolo HTTP/REST para comunicação entre aplicativo e backend;
- CMS Strapi, com banco de dados SQLite, para armazenamento e cálculo do plano alimentar;
- Aplicativo mobile desenvolvido em MIT App Inventor.

6.2 Dificuldades encontradas

7 Conclusão

Esta seção deve ser finalizada Apresentar uma avaliação final do projeto, incluindo resultados, limitações e possíveis melhorias futuras. Atenção: Além dos diagramas da modelagem, é possível acrescentar fotos de protótipos, prints de telas de aplicativo/aplicação, print de simulador, etc.

O desenvolvimento do Xanim's Vault permitiu explorar, de forma prática, os desafios de integração entre um sistema embarcado (ESP32), um backend CMS (Strapi) e um broker de mensagens MQTT (Mosquitto), aplicados a um problema real: a alimentação individualizada e segura de felinos em residências com múltiplos animais.

8 Referências

- The 2021 AAHA Nutrition and Weight Management Guidelines for Dogs and Cats.
- Comedouro automático para cães em situação de rua e resgatados por ONGs. Disponível em: <https://joins.emnuvens.com.br/joins/article/view/1026/1069>.
- Alimentador automático para nutrição de gatos com ESP32. UFRGS. Disponível em: <https://lume.ufrgs.br/handle/10183/301126>.
- ComunaPet - Solução inteligente para alimentação de pets através do uso de IoT. Disponível em: <https://releia.ifsertaope.edu.br/jspui/bitstream/123456789/1021/1/TCC%20-%20COMUNAPET%20SOLUCAO%20INTELIGENTE%20PARA%20ALIMENTACAO%20DE%20PETS%20ATRAVES%20DO%20USO%20DE%20IOT.pdf>.
- Dispenser de ração automático com Arduino. Disponível em: http://ric-cps.eastus2.cloudapp.azure.com/bitstream/123456789/20426/1/eletronica_2024_1__arthursimionatochiqini_dispenseralimentadorpetautomatico.pdf.
- Alimentador na UFERSA com ESP32 e MQTT e dashboards. Disponível em: <https://repositorio.ufersa.edu.br/server/api/core/bitstreams/862731d1-a70c-4e71-9db0-696525ea7b9b/content>.
- Coleira com tecnologia NFC. Disponível em: <http://ric-cps.eastus2.cloudapp.azure.com/handle/123456789/31918>.